

Métodos de Ordenação

Insertion Sort



Linguagem C

Selection Sort



ESTRUTURA DE DADOS

Sem Ordenar

23	4	67	-8	21
----	---	----	----	----

0 1 2 3 4

23	4	67	-8	21
----	---	----	----	----

-8	4	67	23	21
----	---	----	----	----

0 1 2 3 4

-8	4	67	23	21
----	---	----	----	----

-8	4	67	23	21
----	---	----	----	----

Métodos de Ordenação

Profa. Dra. Lúcia F. A. Guimarães



Métodos de Ordenação

- **Introdução**

- **Definição:**

- **Ordenar** corresponde ao processo de **rearranjar um conjunto de objetos** de um vetor ou lista em uma **ordem específica**.

- **Objetivo:**

- **facilitar a recuperação** posterior de elementos do conjunto ordenado

- **Exemplos:**

- Listas telefônicas
- Dicionários
- Índices de livros
- Tabelas e arquivos



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação

• Observações

- Os algoritmos trabalham sobre as **estruturas de um arquivo**.
- Apenas uma parte da estrutura, chamada **chave**, é utilizada para **controlar a ordenação**.
- Além da **chave** podem existir **outros componentes** em uma estrutura, que **não têm influência no processo de ordenar**, a não ser pelo fato de que permanecem com a mesma chave.
- O **tamanho** dos outros componentes, entretanto, **pode influenciar na escolha do método ou na forma de implementação de um dado método**.



A **estrutura de dados** é a indicada para representar os elementos a serem ordenados.

Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



- Estrutura



```
struct item
{
    int chave_t;
    /* outros
    componentes */
};
```

4	10	4	7	9	6	8
---	----	---	---	---	---	---



4	4	6	7	8	9	10
---	---	---	---	---	---	----



Sem Ordenar

23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação

- **CrITÉrios de Classificação**

- **Localização dos dados**



- Ordenação **Interna**:

- Todos os dados estão em **memória principal (RAM)**

- Ordenação **Externa**

- Memória principal não cabe todos os dados que são armazenados em **memória secundária (disco)**



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação

• Critérios de Classificação

• Estabilidade

- Um Método é **estável** se a **ordem relativa dos registros com a mesma chave não se altera** após a ordenação
- Um exemplo melhor:
 - se um vetor tiver dois elementos iguais a 22, primeiro um **azul** e depois um **vermelho**, um algoritmo de ordenação estável mantém o 22 **azul** antes do **vermelho**:

Original: 44 55 55 66 77 33 99 **22** 11 **22** 88

Ordenado: 11 **22** **22** 33 44 55 55 66 77 88 99

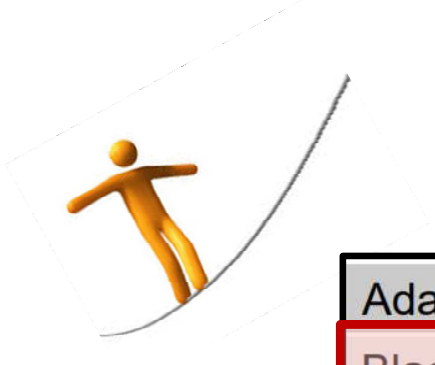


Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



• Critérios de Classificação

• Estabilidade



Adams	A
Black	B
Brown	D
Jackson	B
Jones	D
Smith	A
Thompson	D
Washington	B
White	C
Wilson	C

Adams	A
Smith	A
Washington	B
Jackson	B
Black	B
White	C
Wilson	C
Thompson	D
Brown	D
Jones	D

↳ Chave

Adams	A
Smith	A
Black	B
Jackson	B
Washington	B
White	C
Wilson	C
Brown	D
Jones	D
Thompson	D

↳ Chave



Sem Ordenar

23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação

- **CrITÉrios de Classificação**

- **Adaptabilidade**

- Método **Adaptável**:

- a sequencia de operações realizadas **depende** da entrada

- Método **NÃO Adaptável**

- a sequencia de operações é a mesma **independente** da entrada.



Sem Ordenar

23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



- **Critérios de Classificação**

- **Usa da Memória**

- ***"In place"***: ordena sem usar memória adicional ou usando uma quantidade constante de memória adicional.

- Alguns métodos precisam duplicar os dados



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação



- **Crítérios de Classificação**
 - Movimentação dos dados

```
// struct item a;  
// struct item b;  
struct item aux = a;  
a = b;  
b = aux;
```

- **Direta:** a estrutura toda é movida

- **Indireta:** apenas as chaves são acessadas e ponteiros para as estruturas são rearranjados

```
// struct item *a;  
// struct item *b;  
struct item *aux = *a;  
a = b;  
b = aux;
```



Sem Ordenar

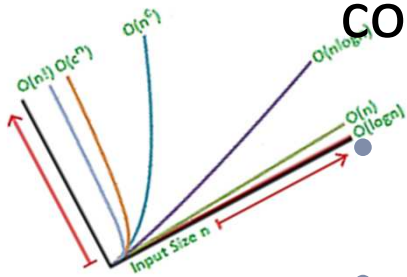
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



- **Critérios de Avaliação**

- **Medidas de Complexidade**

- Seja **n** o número de registros em um vetor, considera-se **duas medidas de complexidade**:



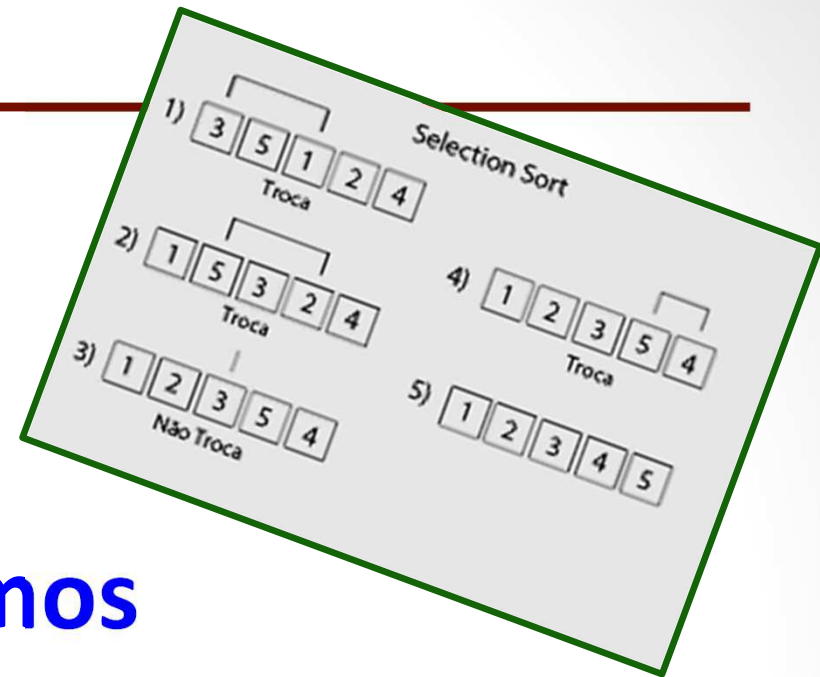
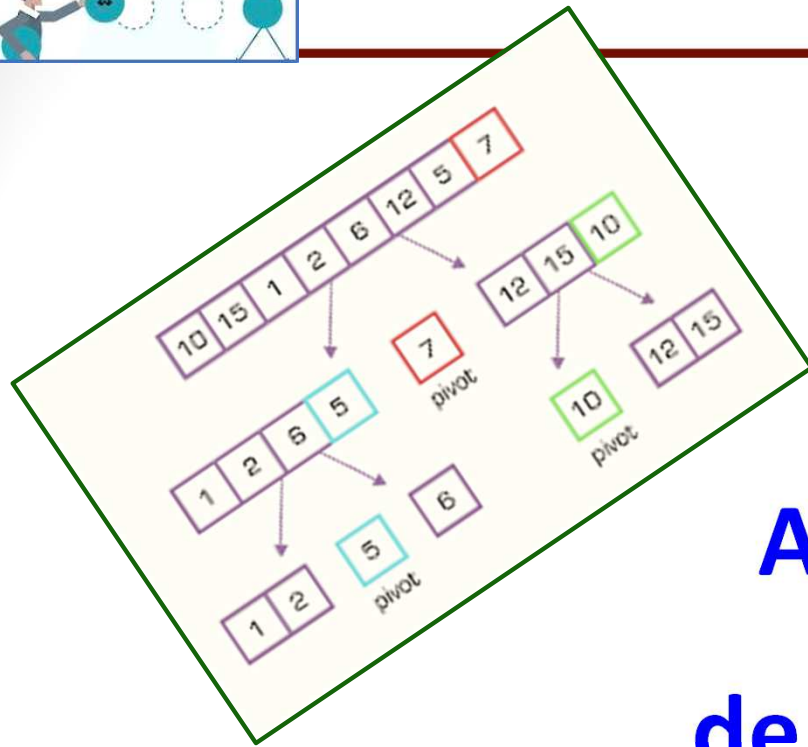
- **Número de comparações $C(n)$ entre as chaves**

- **Número de trocas ou movimentações $M(n)$ de itens.**

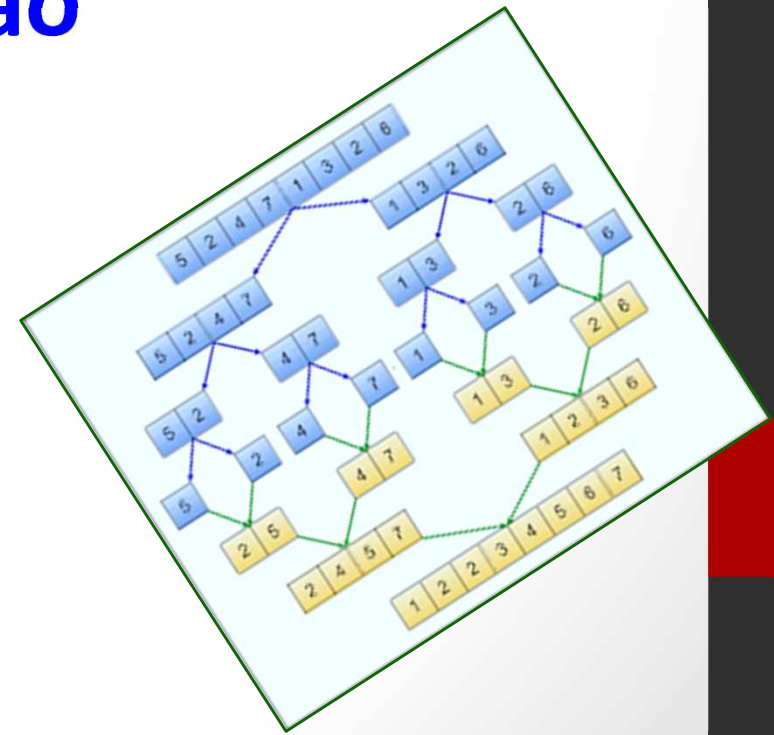
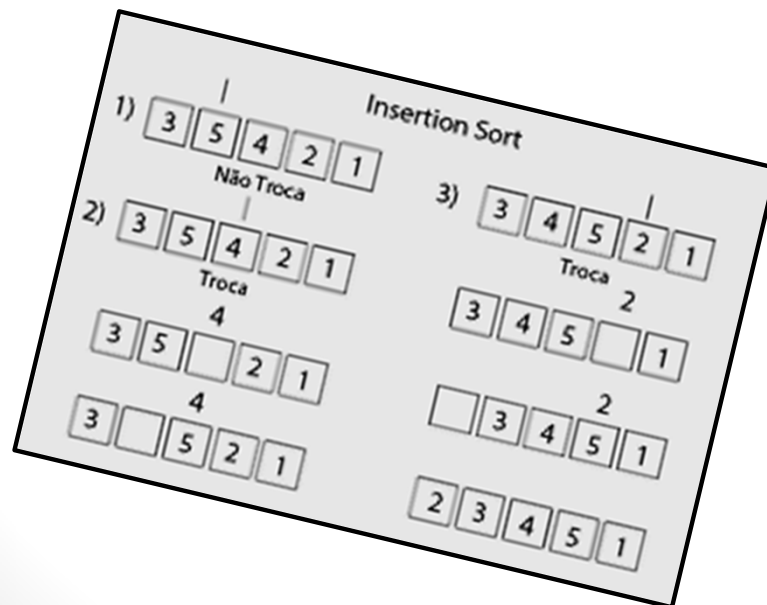
- **Não vamos discutir esse tópico nessa disciplina**



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Algoritmos de Ordenação



Selection Sort

Selection Sort
Ordenação por Seleção

-1	9	4	40	6			
	-1	4	9	40	6		
		-1	4	6	40	9	
			-1	4	6	9	40



Métodos de Ordenação- Selection Sort

- **“Selection Sort” – Ordenação por Seleção**



<https://youtu.be/0-W8OEwLebQ?si=fGtSUwlzgOk6a6-4>

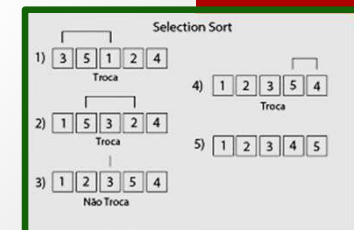


Métodos de Ordenação- Selection Sort

- **“Selection Sort” – Ordenação por Seleção**



- Um dos algoritmos **mais simples**.
- Princípio:
 1. **Procura** o n-ésimo menor elemento do vetor
 2. **Troca** do n-ésimo menor elemento com o elemento na n-ésima posição
 3. **Repete** até ter colocado todos os elementos em suas posições
- **Elementos são movimentados apenas uma vez**



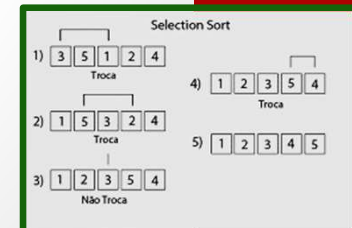


Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção
- Algoritmo



```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



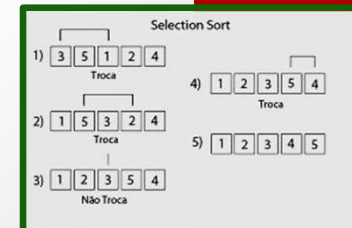
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 0$

$j = 1, \dots, 5$

0	1	2	3	4	5
44	12	55	42	94	18





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção

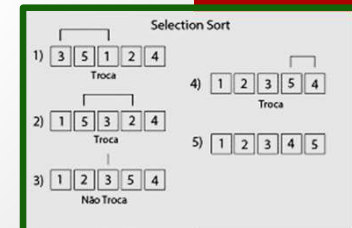


- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 0$
 $min = 1$

0	1	2	3	4	5
44	12	55	42	94	18





Métodos de Ordenação- Selection Sort

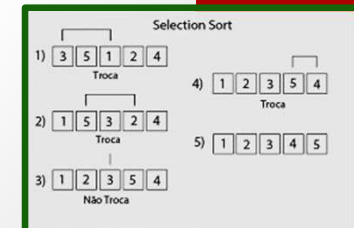
- “Selection Sort” – Ordenação por Seleção



- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

0	1	2	3	4	5
12	44	55	42	94	18





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



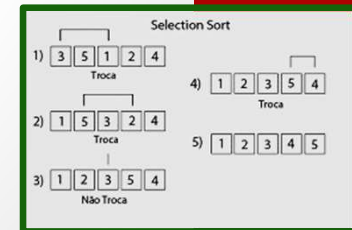
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 1$

$j = 2, \dots, 5$

0	1	2	3	4	5
12	44	55	42	94	18





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção

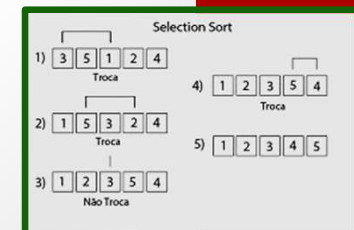


- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 1$
 $min = 5$

0	1	2	3	4	5
12	44	55	42	94	18





Métodos de Ordenação- Selection Sort

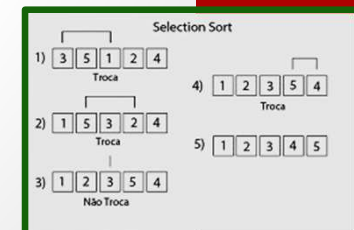
- “Selection Sort” – Ordenação por Seleção



- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

0	1	2	3	4	5
12	18	55	42	94	44





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



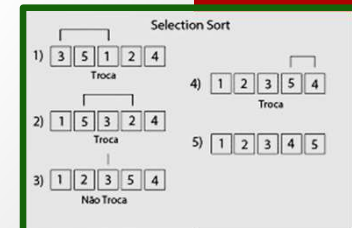
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 2$

$j = 3, \dots, 5$

0	1	2	3	4	5
12	18	55	42	94	44





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



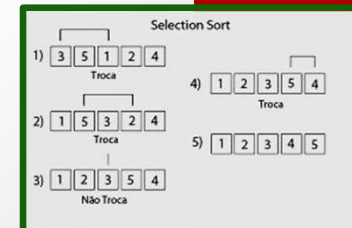
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 2$

$min = 3$

0	1	2	3	4	5
12	18	55	42	94	44





Métodos de Ordenação- Selection Sort

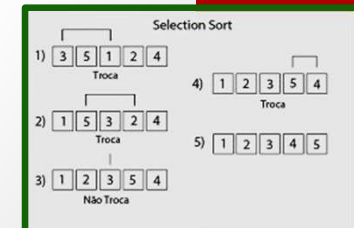
- “Selection Sort” – Ordenação por Seleção



- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

0	1	2	3	4	5
12	18	42	55	94	44





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



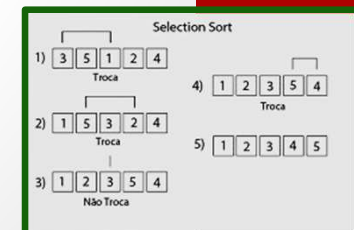
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 3$

$j = 4, 5$

0	1	2	3	4	5
12	18	42	55	94	44





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção

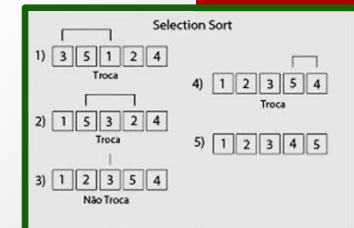


- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 3$
 $min = 5$

0	1	2	3	4	5
12	18	42	55	94	44





Métodos de Ordenação- Selection Sort

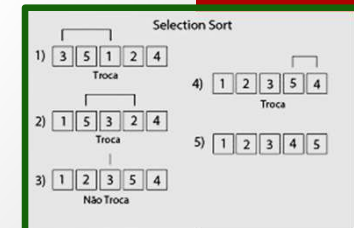
- “Selection Sort” – Ordenação por Seleção



- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

0	1	2	3	4	5
12	18	42	44	94	55





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



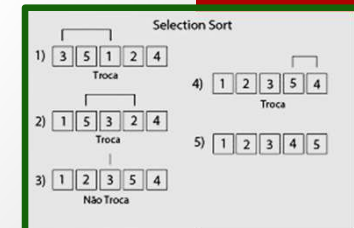
- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 4$

$j = 5$

0	1	2	3	4	5
12	18	42	44	94	55





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção

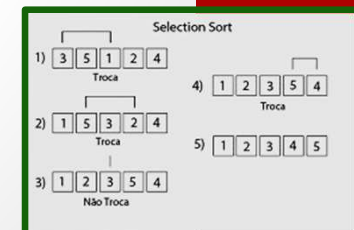


- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

$i = 4$
 $min = 5$

0	1	2	3	4	5
12	18	42	44	94	55





Métodos de Ordenação- Selection Sort

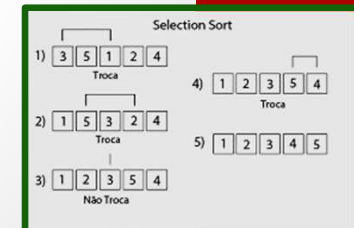
- “Selection Sort” – Ordenação por Seleção



- Exemplo

```
void selecao(struct item *v, int n){  
    int i, j, min;  
    for(i = 0; i < n - 1; i++) {  
        min = i;  
        for(j = i + 1 ; j < n; j++) {  
            if(v[j].chave < v[min].chave)  
                min = j;  
        }  
        troca(v[i], v[min]);  
    }  
}
```

0	1	2	3	4	5
12	18	42	44	55	94





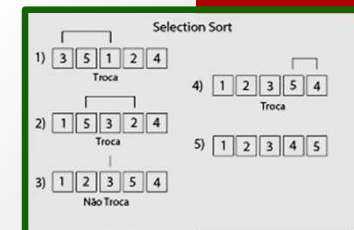
Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



- Vantagens

- É um dos métodos **mais simples** de ordenação existentes.
- Para arquivos com **registros muito grandes e chaves pequenas**, este deve ser o método a ser utilizado.
- Com **chaves do tamanho de uma palavra**, este método torna-se bastante interessante para **arquivos pequenos**.





Métodos de Ordenação- Selection Sort

- “Selection Sort” – Ordenação por Seleção



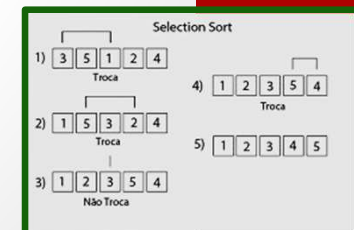
- Desvantagem

- **NÃO** é adaptável

- Mesmo que o arquivo já esteja ordenado não ajuda em nada pois o custo continua quadrático

- **NÃO** é estável

- Nem sempre deixa os registros com chaves iguais na mesma posição relativa.



Insertion Sort

7	4	5	9	8	2	1
---	---	---	---	---	---	---

4	7	5	9	8	2	1
---	---	---	---	---	---	---

4	5	7	9	8	2	1
---	---	---	---	---	---	---

4	5	7	8	9	2	1
---	---	---	---	---	---	---

2	4	5	7	8	9	1
---	---	---	---	---	---	---

1	2	4	5	7	8	9
---	---	---	---	---	---	---

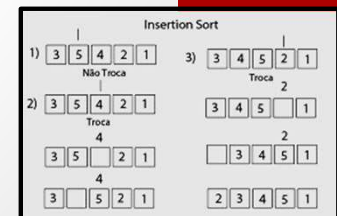
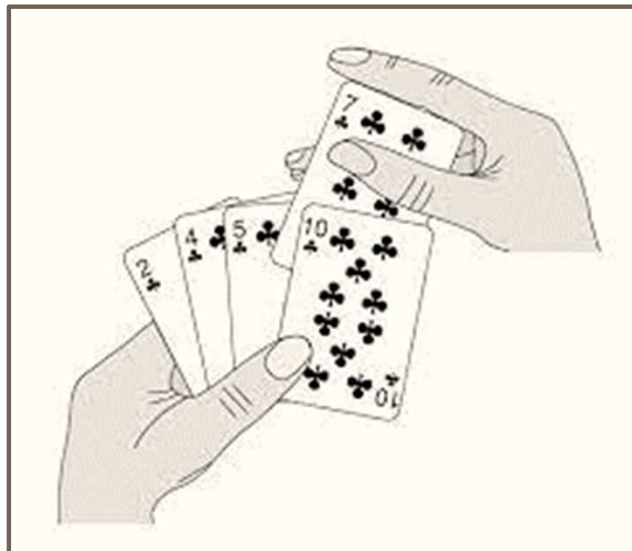


Métodos de Ordenação- Insertion Sort

• “Insertion Sort” – Ordenação por Inserção



- Algoritmo utilizado pelo jogador de cartas
 - Jogador mantém as cartas em sua mão ordenadas
 - Quando compra ou recebe uma nova carta, o jogador encontra qual posição ela deve ocupar em sua mão.





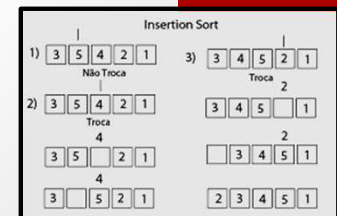
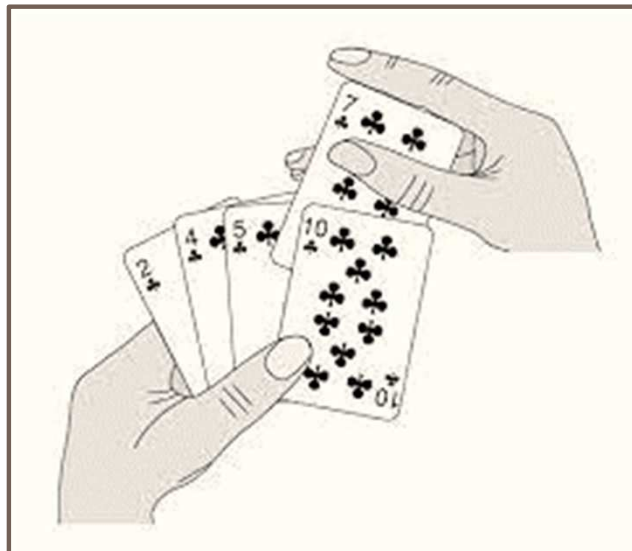
Métodos de Ordenação- Insertion Sort

• “Insertion Sort” – Ordenação por Inserção



• Princípio:

- Em cada passo, a partir de $i = 1$,
 - o i -ésimo elemento da sequência fonte é apanhado e
 - transferido para a sequência destino, sendo inserido no seu lugar apropriado.



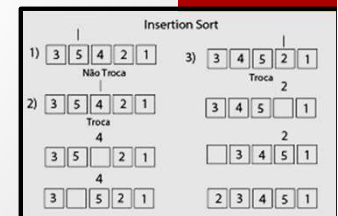


Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção



<https://youtu.be/EdIKIf9mHk0?si=EI56hLe069XAkpzd>





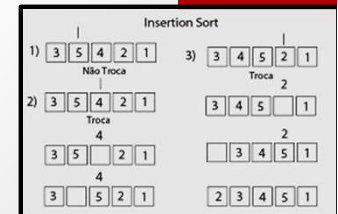
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção



- Algoritmo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```





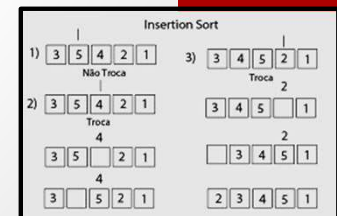
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
44	12	55	42	94	18





Métodos de Ordenação- Insertion Sort

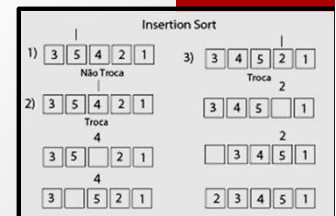
- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

$i = 1$
 $j = 1, 0$

0	1	2	3	4	5
44	12	55	42	94	18





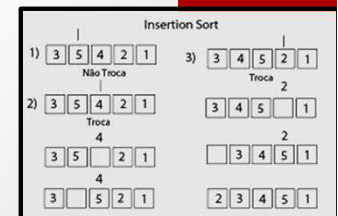
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
12	44	55	42	94	18





Métodos de Ordenação- Insertion Sort

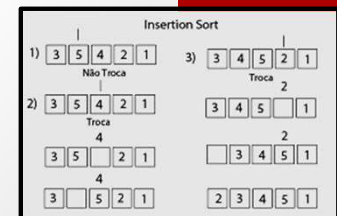
- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

$i = 2$
 $j = 2, 1, 0$

0	1	2	3	4	5
12	44	55	42	94	18





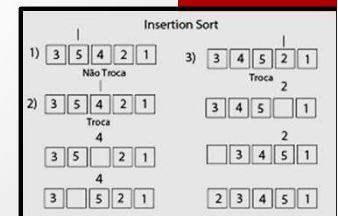
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
12	44	55	42	94	18





Métodos de Ordenação- Insertion Sort

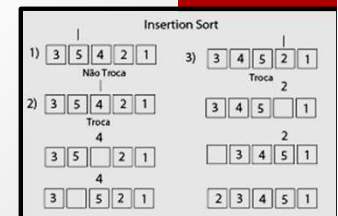
- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

$i = 3$
 $j = 3, 2, 1, 0$

0	1	2	3	4	5
12	44	55	42	94	18





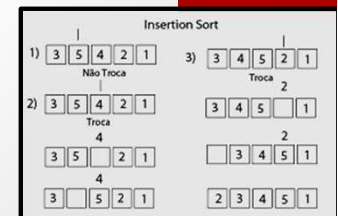
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
12	42	44	55	94	18



Métodos de Ordenação- Insertion Sort

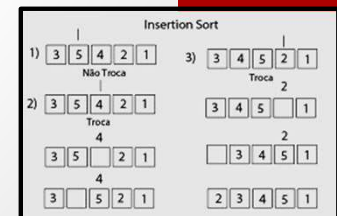
- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

$i = 4$
 $j = 4, 3, 2, 1, 0$

0	1	2	3	4	5
12	42	44	55	94	18





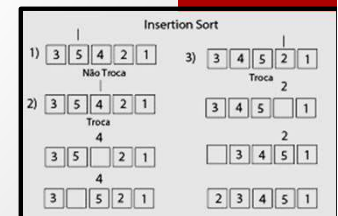
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
12	42	44	55	94	18





Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

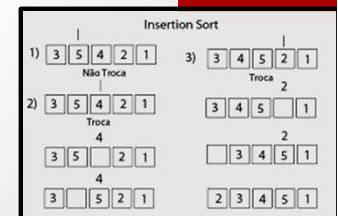
- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

$i = 5$

$j = 5, 4, 3, 2, 1, 0$

0	1	2	3	4	5
12	42	44	55	94	18





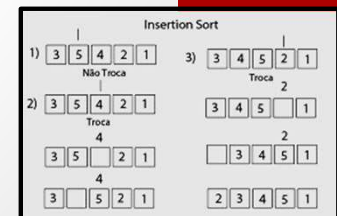
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção

- Exemplo

```
void insercao(struct item *v, int n) {  
    int i, j;  
    struct item aux;  
  
    for(i = 1; i < n; i++) {  
        for(j = i; j >= 0; j--) {  
            if(v[j-1].chave > v[j].chave) {  
                troca(v[j-1], v[j]);  
            }  
        }  
    }  
}
```

0	1	2	3	4	5
12	18	42	44	55	94





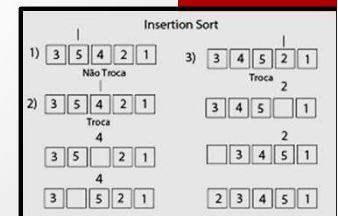
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção



- Vantagens

- Laço interno é eficiente, inserção é adequado para **ordenar vetores pequenos**
- É o método a ser utilizado quando **o arquivo está “quase” ordenado**
- É um bom método quando se deseja **adicionar poucos itens a um arquivo ordenado**, pois o custo é linear
- É um **algoritmo estável**, pois deixa os registros **com chaves iguais** na mesma posição relativa.





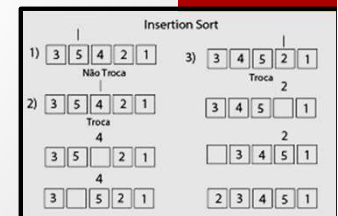
Métodos de Ordenação- Insertion Sort

- “Insertion Sort” – Ordenação por Inserção



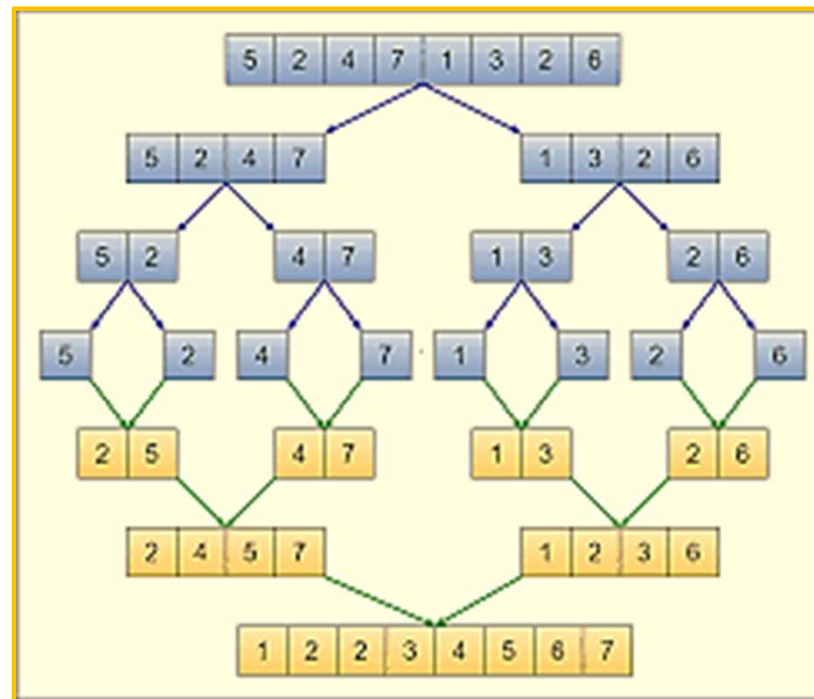
- Desvantagens

- Número de comparações tem **crescimento quadrático**
- Alto custo de movimentação de elementos no vetor





Merge Sort



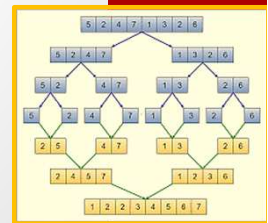


Métodos de Ordenação Merge Sort

• Introdução



- O algoritmo Merge Sort é um algoritmo do tipo **dividir para conquistar**
- O processo-chave do Merge Sort consiste em
 - **dividir o vetor original** em sub-vetores cada vez menores
 - até que se tenha pequenos **sub-vetores com um elemento apenas.**
- A partir daí,
 - cada par de sub-vetores é “fundido (merged)” de forma intercalada,
 - até se obter um único vetor ordenado com todos os elementos.



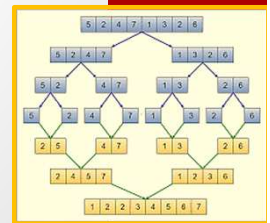


Métodos de Ordenação Merge Sort

- **Motivação**



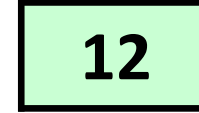
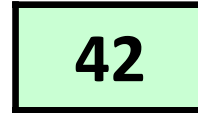
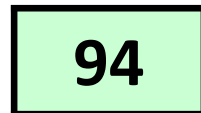
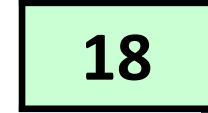
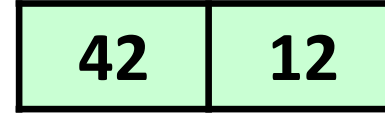
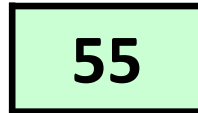
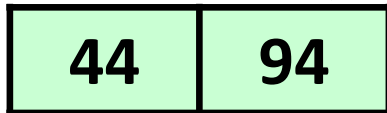
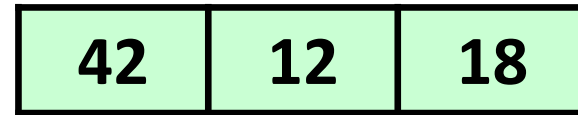
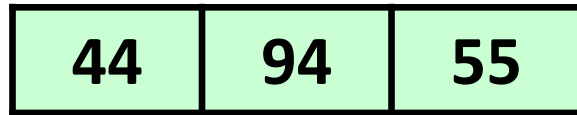
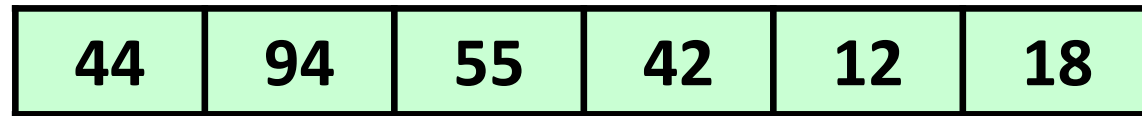
- Técnica Dividir para Conquistar:
 1. **Divisão:** Dividir o problema original em subproblemas menores recursivamente até que o subproblema possua um elemento
 2. **Conquista:** Resolver cada subproblema
 3. **Merge (Combinação):** Combinar as soluções encontradas, compondo uma solução para o problema original



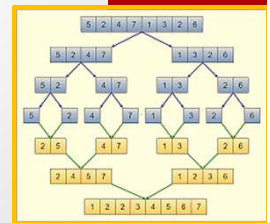


Métodos de Ordenação Merge Sort

- Exemplo



DIVISÃO

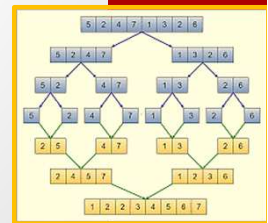
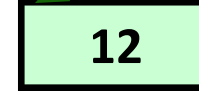
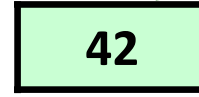
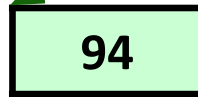
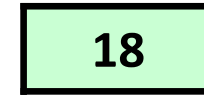
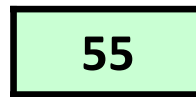
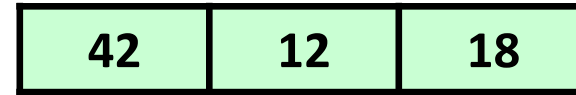
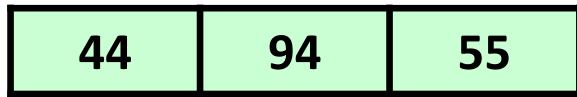




CONQUISTA

nação Merge Sort

- Exemplo

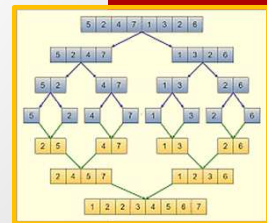
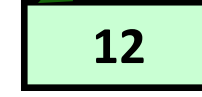
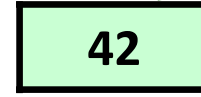
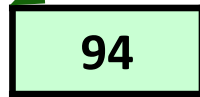
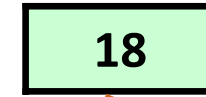
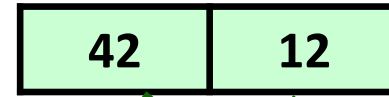
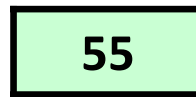
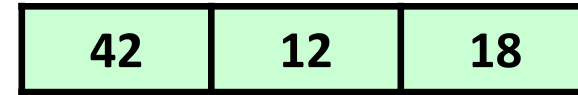
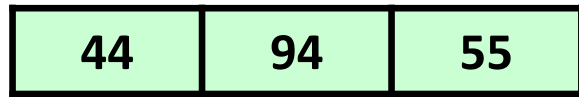




CONQUISTA

nação Merge Sort

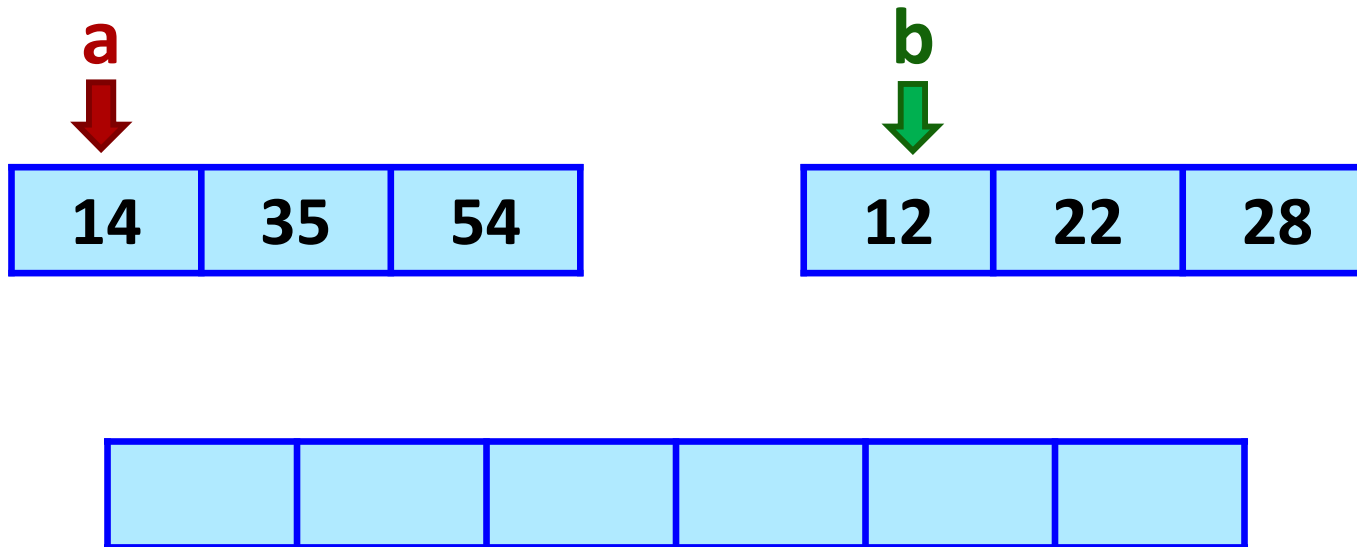
- Exemplo



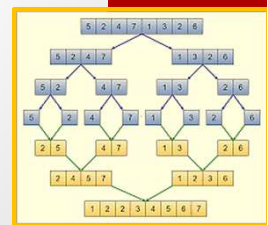


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



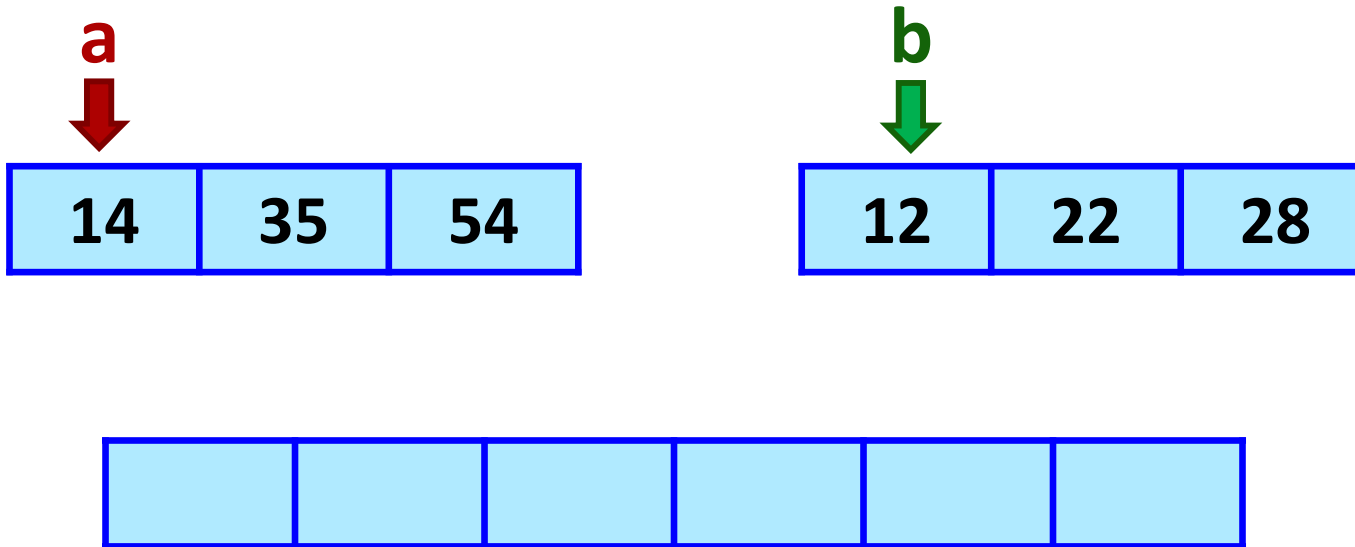
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

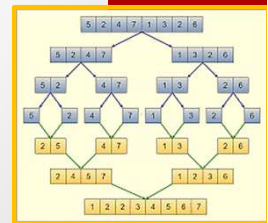
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

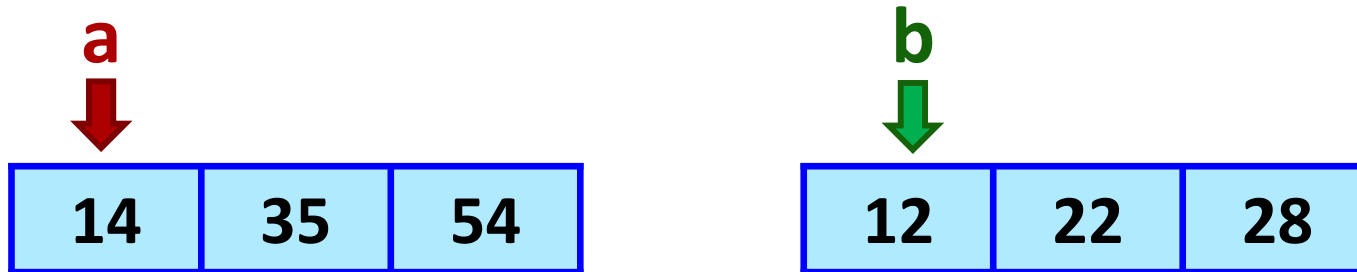
incrementa **b**



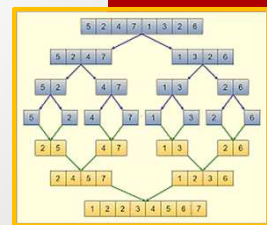


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



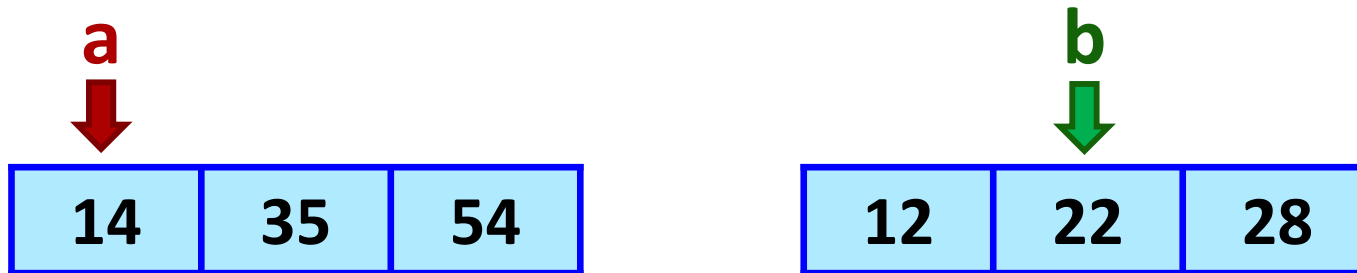
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**



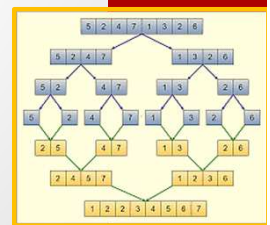


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



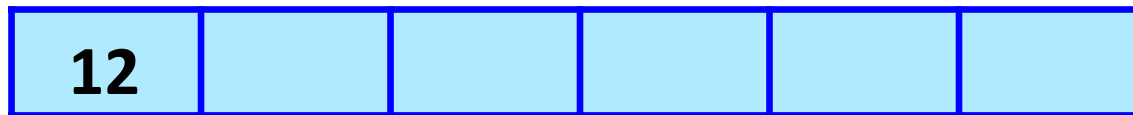
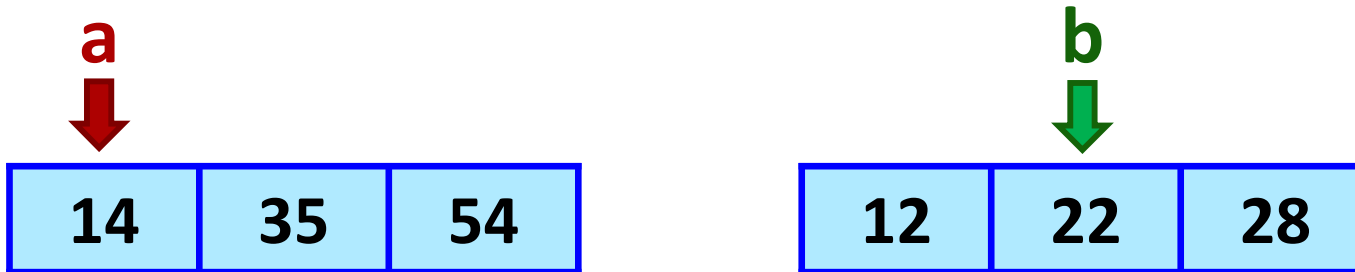
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

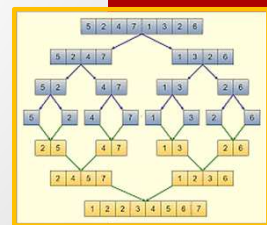
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

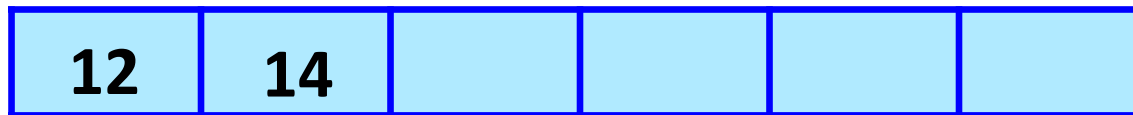
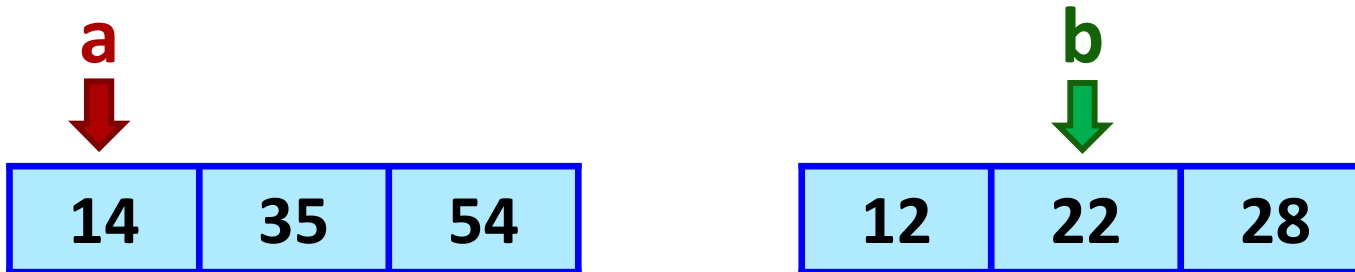
incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

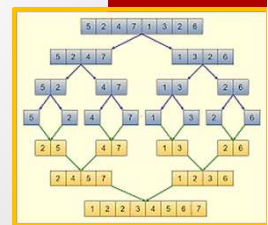
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

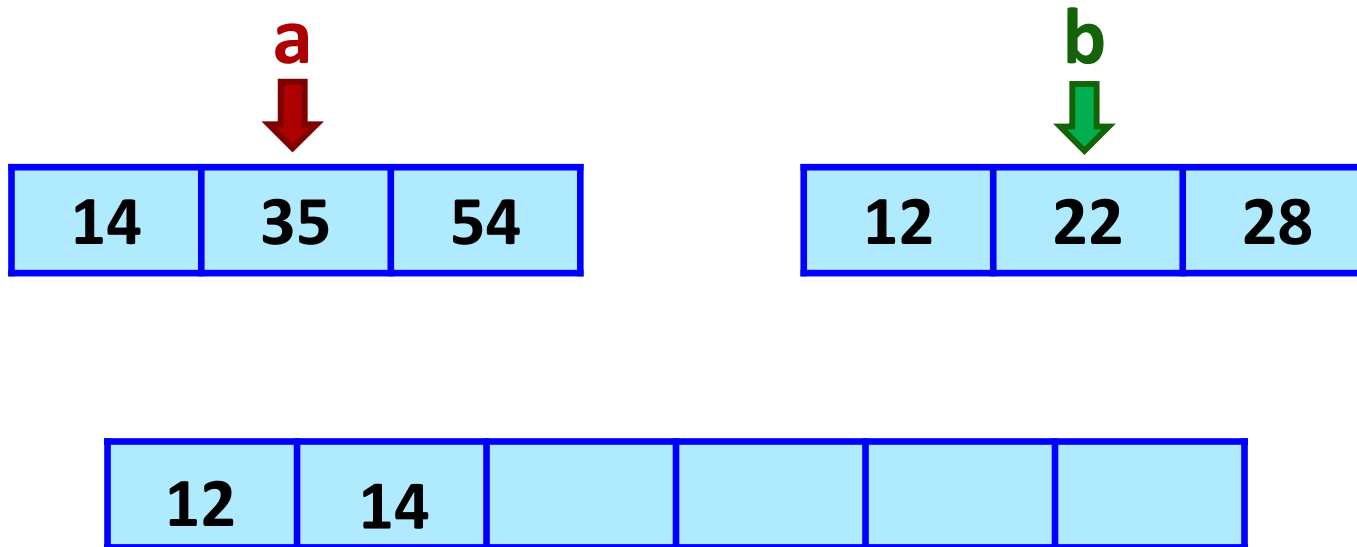
incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

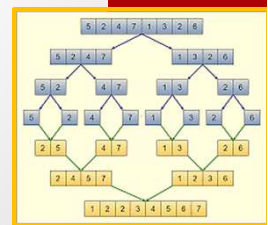
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

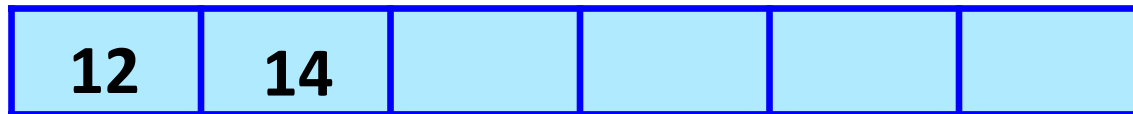
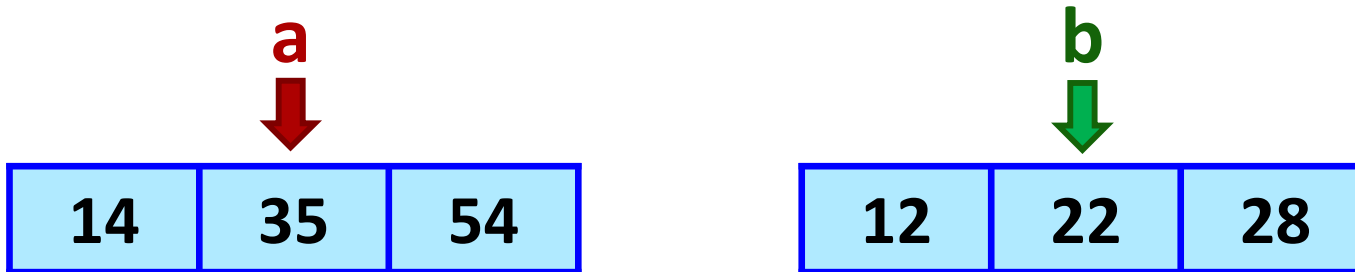
incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

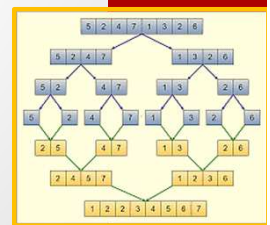
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

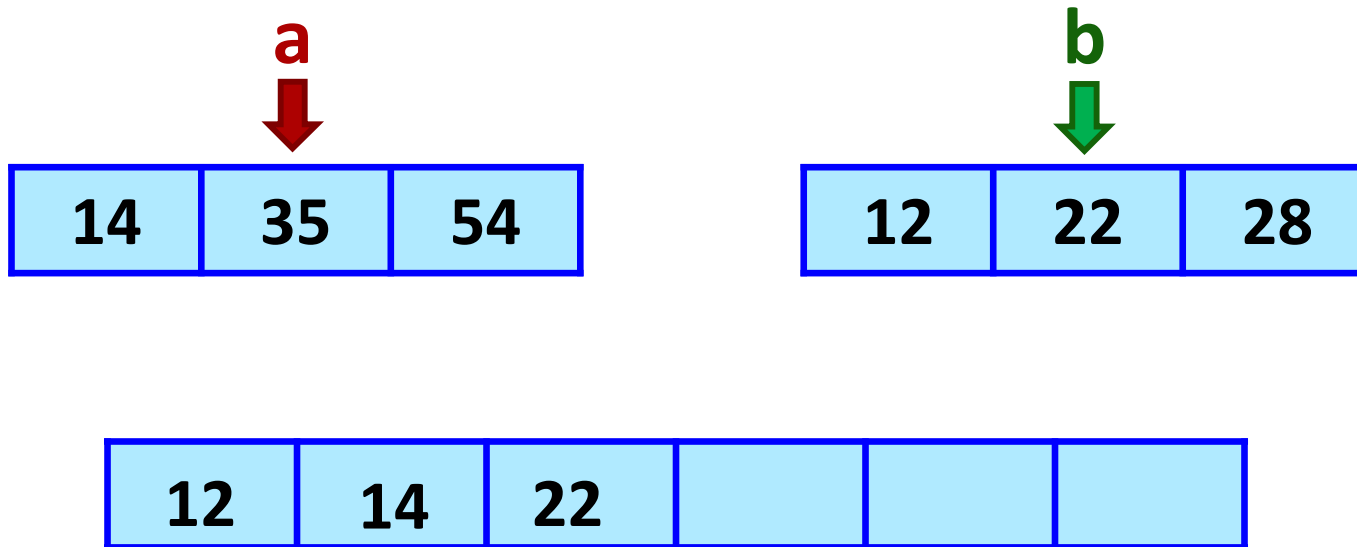
incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

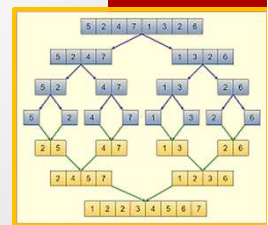
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

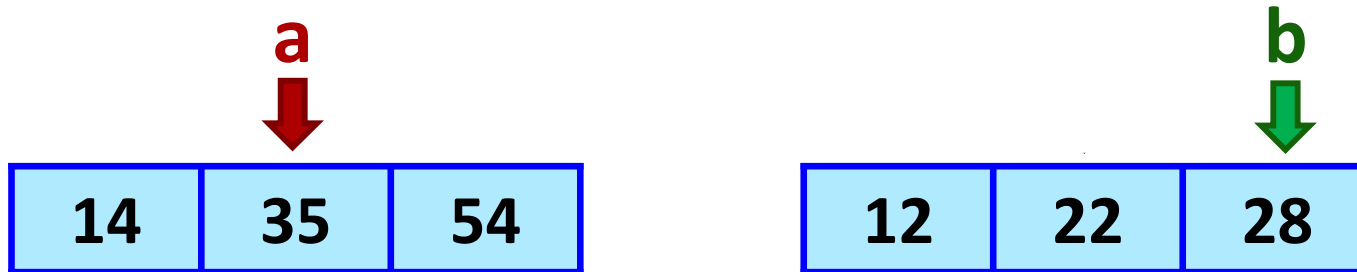
incrementa **b**



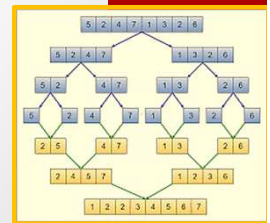


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



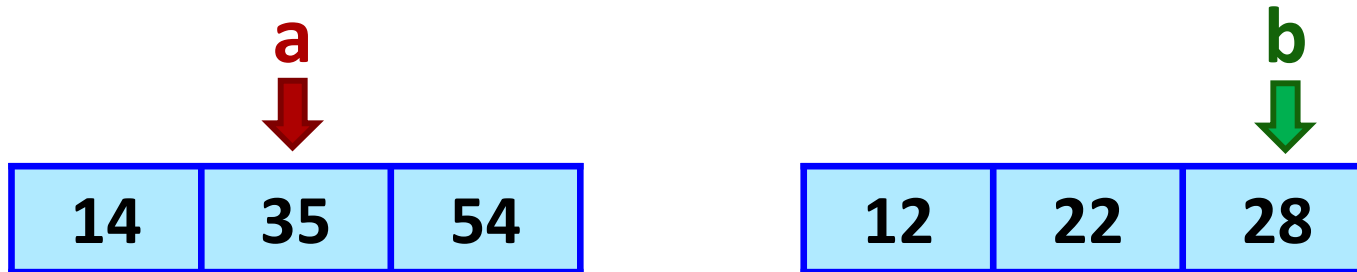
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então

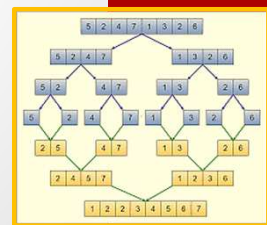
insere $v[a]$

incrementa **a**

senão

insere $v[b]$

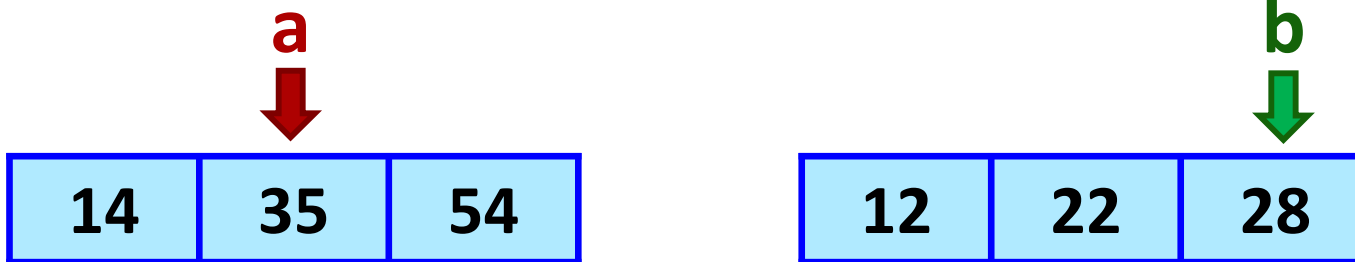
incrementa **b**



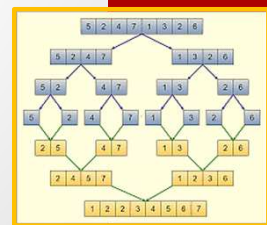


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



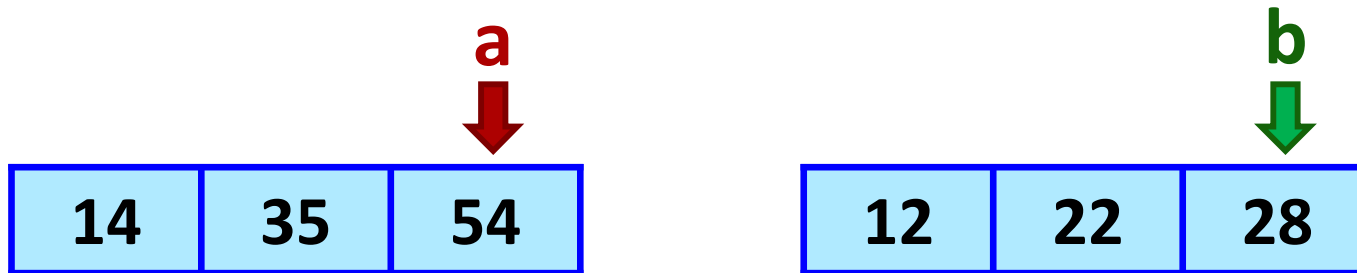
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**



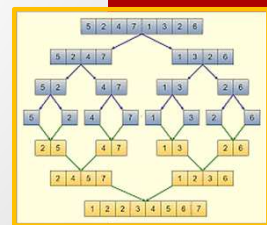


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



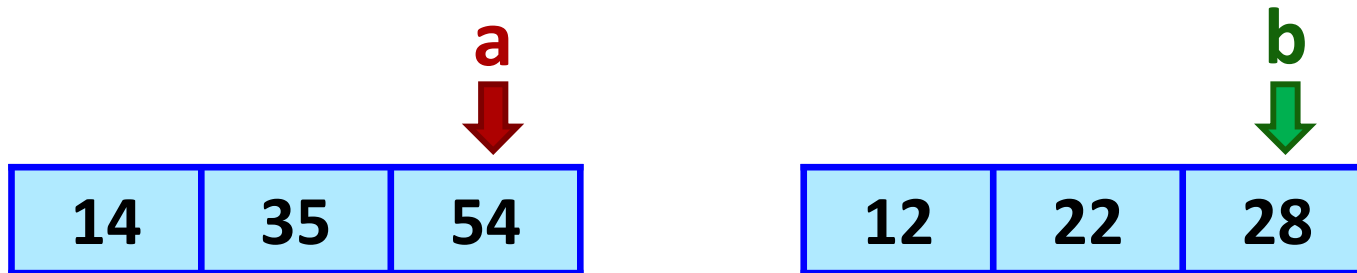
Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**



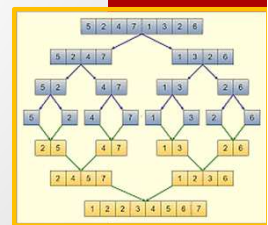


Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Se $v[a] < v[b]$ então
 insere $v[a]$
 incrementa **a**
senão
 insere $v[b]$
 incrementa **b**





Métodos de Ordenação Merge Sort



```
void mergesort(int *v, int n)
{
    int *c = malloc(sizeof(int) * n);
    sort(v, c, 0, n - 1);
    free(c);
}
```

O Merge Sort é chamado passando:

- O vetor v
- E o tamanho de v, n



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação Merge Sort



```
void mergesort(int *v, int n)
{
    int *c = malloc(sizeof(int) * n);
    sort(v, c, 0, n - 1);
    free(c);
}
```

Um novo espaço é alocado para um vetor c

Observe que c tem o tamanho do vetor v, n



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação Merge Sort



```
void mergesort(int *v, int n)
{
    int *c = malloc(sizeof(int) * n);
    sort(v, c, 0, n - 1);
    free(c);
}
```

O procedimento *sort* é chamado passando como parâmetros:

- vetor *v* e o vetor *c*
- O limite inferior do vetor (0)
- O limite superior do vetor (n-1)



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação Merge Sort



```
void mergesort(int *v, int n)
{
    int *c = malloc(sizeof(int) * n);
    sort(v, c, 0, n - 1);
    free(c);
}
```

Logo após a execução do procedimento *sort*, o vetor *c* que havia sido alocado é liberado



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação Merge Sort



```
void sort(int *v, int *c, int i, int f)
{
    if (i < f)
    {
        int m = (i + f) / 2;
        sort(v, c, i, m);
        sort(v, c, m + 1, f);
        if(v[m] > v[m + 1])
        {
            merge(v, c, i, m, f);
        }
    }
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



```
void sort(int *v, int *c, int i, int f)
{
    if (i < f)
    {
        int m = (i + f) / 2;
        sort(v, c, i, m);
        sort(v, c, m + 1, f);
        if(v[m] > v[m + 1])
        {
            merge(v, c, i, m, f);
        }
    }
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



```
void sort(int *v, int *c, int i, int f)
{
    if (i < f)
    {
        int m = (i + f) / 2;
        sort(v, c, i, m);
        sort(v, c, m + 1, f);
        if (v[m] > v[m + 1])
        {
            merge(v, c, i, m, f);
        }
    }
}
```



Sem Ordenar				
23	4	67	-8	21
0	1	2	3	4
23	4	67	-8	21
-8	4	67	23	21
0	1	2	3	4
-8	4	67	23	21
-8	4	67	23	21



Métodos de Ordenação Merge Sort



```
void sort(int *v, int *c, int i, int f)
{
    if (i < f)
    {
        int m = (i + f) / 2;
        sort(v, c, i, m);
        sort(v, c, m + 1, f)
        if(v[m] > v[m + 1])
        {
            merge(v, c, i, m, f);
        }
    }
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



```
void merge(int *v, int *c, int i, int m, int f)
```

```
{  
    int z, iv = i, ic = m + 1;  
    for (z = i; z <= f; z++) c[z] = v[z];  
    z = i;  
    while (iv <= m && ic <= f)  
    {  
        if (c[iv] <= c[ic])  
        {  
            v[z++] = c[iv++];  
        }  
        else  
        {  
            v[z++] = c[ic++];  
        }  
    }  
    while (iv <= m)  
    {  
        v[z++] = c[iv++];  
    }  
    while (ic <= f)  
    {  
        v[z++] = c[ic++];  
    }  
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



```
void merge(int *v, int *c, int i, int m, int f)
{
    int z, iv = i, ic = m + 1;
    for (z = i; z <= f; z++) c[z] = v[z];
    z = i;
    while (iv <= m && ic <= f)
    {
        if (c[iv] <= c[ic])
        {
            v[z++] = c[iv++];
        }
        else
        {
            v[z++] = c[ic++];
        }
    }
    while (iv <= m)
    {
        v[z++] = c[iv++];
    }
    while (ic <= f)
    {
        v[z++] = c[ic++];
    }
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



```
void merge(int *v, int *c, int i, int m, int f)
{
    int z, iv = i, ic = m + 1;
    for (z = i; z <= f; z++) c[z] = v[z];
    z = i;
    while (iv <= m && ic <= f)
    {
        if (c[iv] <= c[ic])
        {
            v[z++] = c[iv++];
        }
        else
        {
            v[z++] = c[ic++];
        }
    }
    while (iv <= m)
    {
        v[z++] = c[iv++];
    }
    while (ic <= f)
    {
        v[z++] = c[ic++];
    }
}
```



Sem Ordenar					
23	4	67	-8	21	
0	1	2	3	4	
23	4	67	-8	21	
-8	4	67	23	21	
0	1	2	3	4	
-8	4	67	23	21	
-8	4	67	23	21	



Métodos de Ordenação Merge Sort



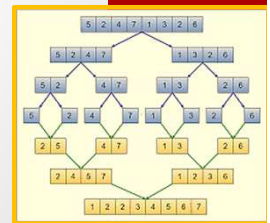
- **Vantagens**

- O Merge Sort

- É um método eficiente e **não sensível à ordem** em que os elementos aparecem no vetor, e
- É **estável**

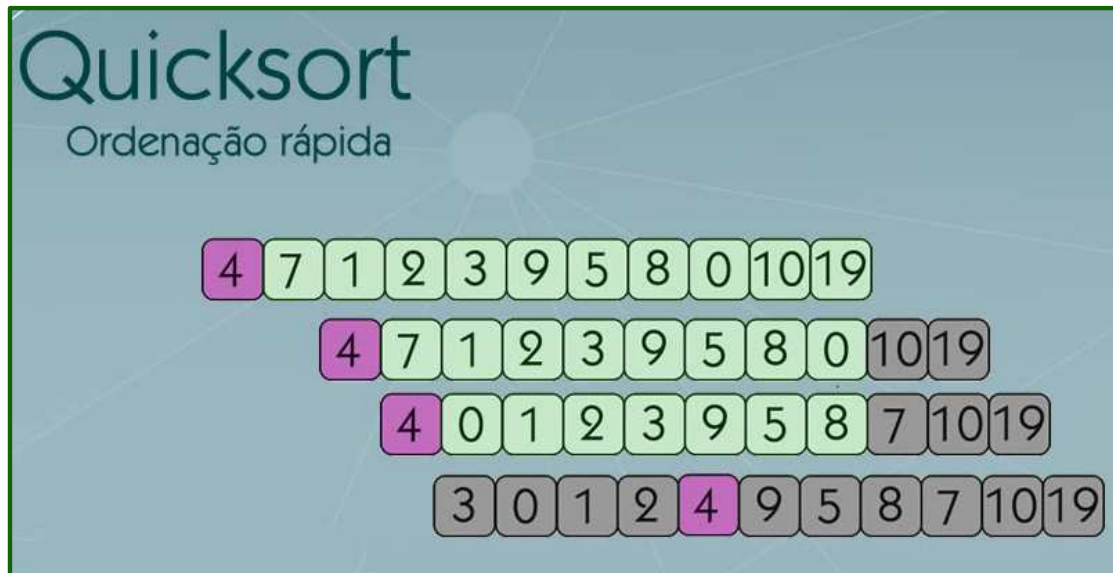
- **Desvantagens**

- O Merge Sort **não é in-place**, requer uma quantidade de espaço extra proporcional ao tamanho do vetor.





Quick Sort





Métodos de Ordenação – Quick Sort

- **Introdução:**

- Proposto por Hoare em 1960 e publicado em 1962
- É o algoritmo de ordenação interna mais rápido que se conhece para uma ampla variedade de situações
- Provavelmente é o mais utilizado





Métodos de Ordenação – Quick Sort

- **Idéia Básica**



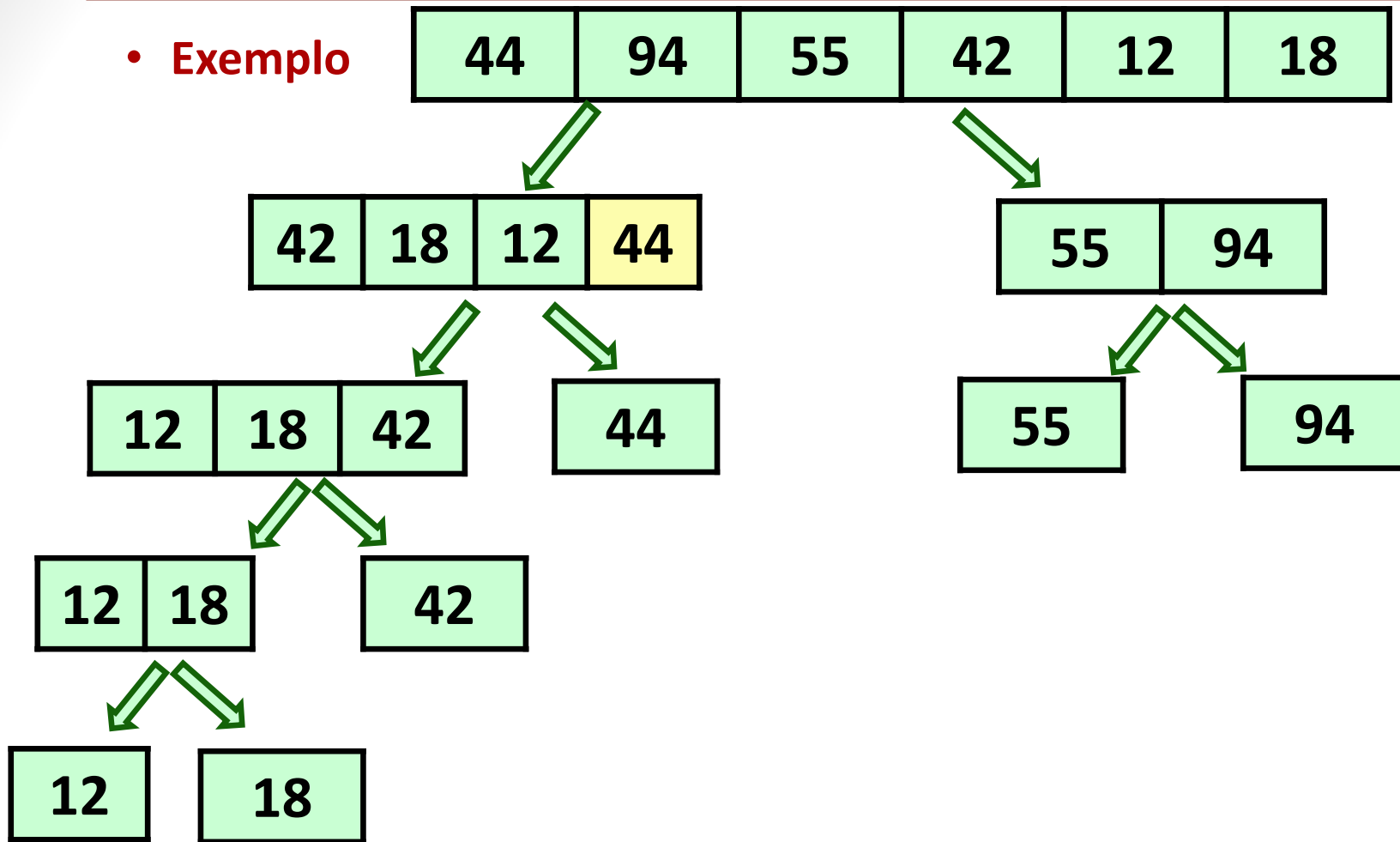
- Dentre os elementos do vetor escolha um que será o pivô
- Coloca-se o pivô na posição correta no vetor
- Garantindo que todos os elementos antes do pivô são menores que ele e todos os elementos depois do pivô são maiores que ele.
- O vetor então é subdividido em 2 um da posição inicial do vetor até o pivô e o outro do pivô até a posição final do vetor e o processo é repetido





Métodos de Ordenação Quick Sort

- Exemplo



DIVISÃO



Quicksort
Ordenação rápida

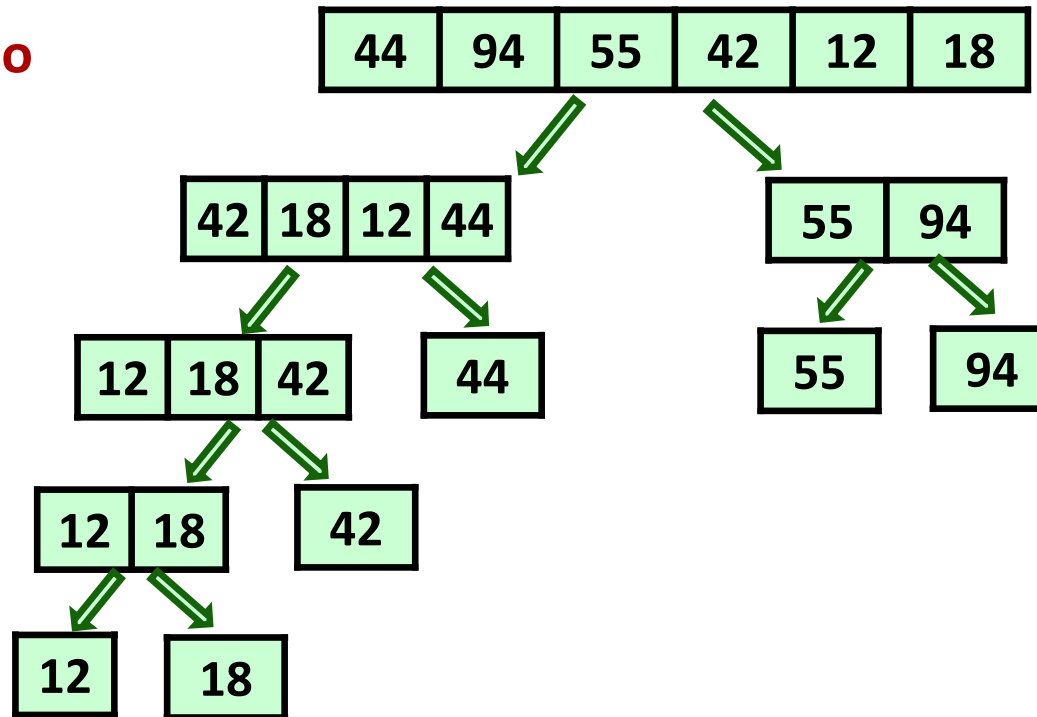
4	7	1	2	3	9	5	8	0	10	19
4	7	1	2	3	9	5	8	0	10	19
4	0	1	2	3	9	5	8	7	10	19
3	0	1	2	4	9	5	8	7	10	19



CONQUISTA

Ordenação Quick Sort

- Exemplo



Quicksort
Ordenação rápida

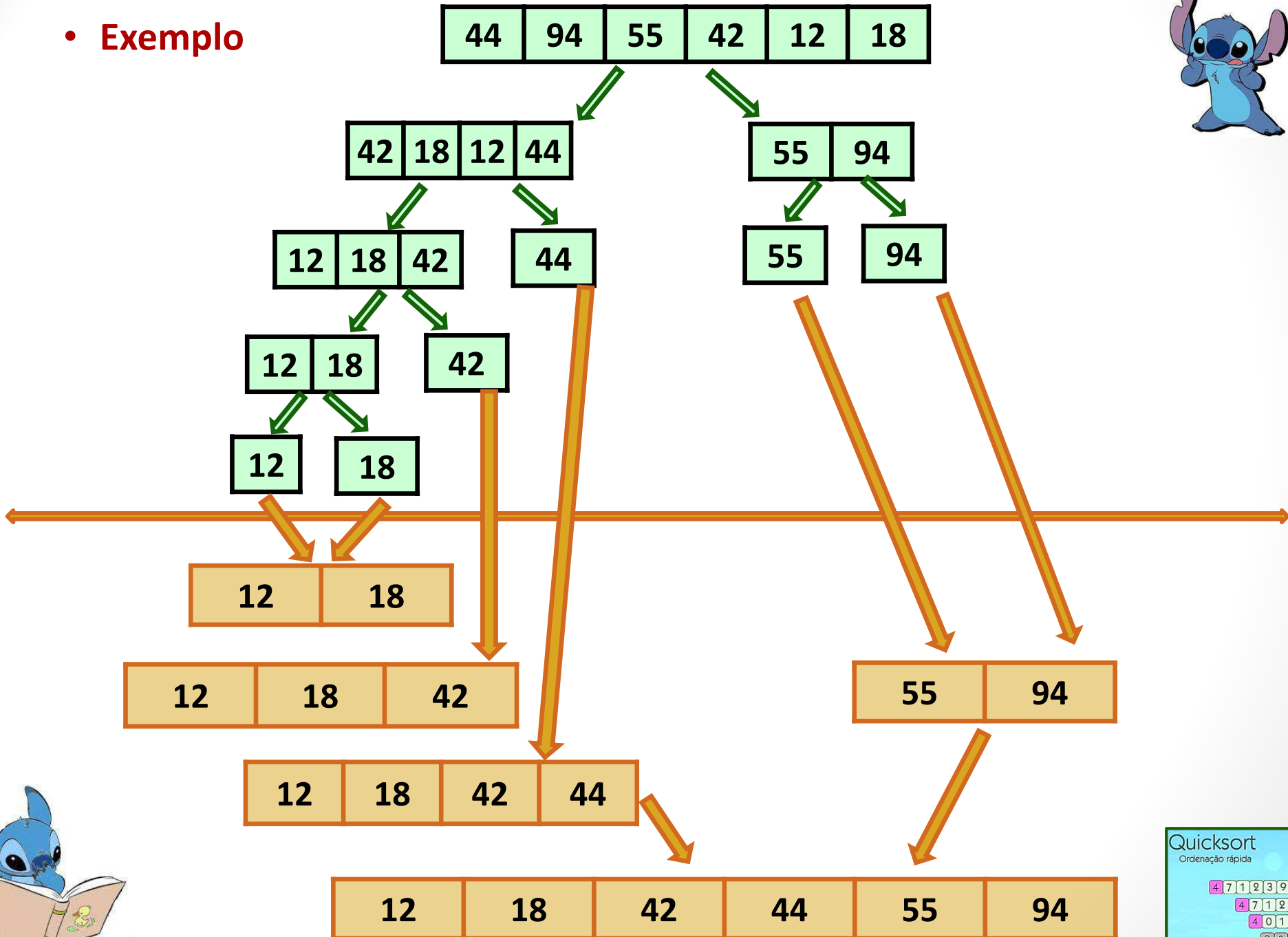
4	7	1	2	3	9	5	8	0	10	19
4	7	1	2	3	9	5	8	0	10	19
4	0	1	2	3	9	5	8	7	10	19
3	0	1	2	4	9	5	8	7	10	19



CONQUISTA

Ordenação Quick Sort

- Exemplo





Métodos de Ordenação – Quick Sort

- **Particionamento**



- **Parte mais delicada** do Quick-Sort é o processo de partição
- O vetor **V** é rearranjado por meio da escolha arbitrária de um pivô **P**
- O vetor **V** é particionado em dois:
 - Partição esquerda: chaves $< \mathbf{P}$
 - Partição direita: chaves $> \mathbf{P}$

**Principal Ideia da
Partição no Quick
Sort**





- **O Algoritmo para o Particionamento**



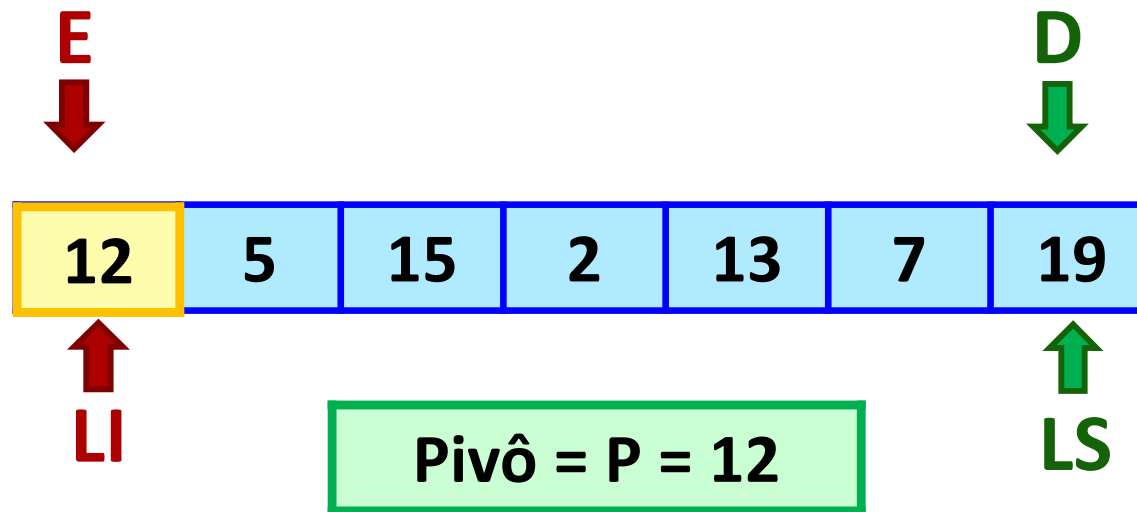
- Escolha arbitrariamente o pivô **P**
- Percorra a partir da esquerda até que $v[\mathbf{E}] > \mathbf{P}$
- Percorra a partir da direita até que $v[\mathbf{D}] < \mathbf{P}$
- Troque $v[\mathbf{E}]$ com $v[\mathbf{D}]$
- Repita os passos anteriores até que **E** e **D** se cruzarem, ou seja, $(\mathbf{D} \leq \mathbf{E})$





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



1. Escolha do pivô ($\text{Pivô} = v[E]$)

2. Enquanto $v[E] \leq P$ INCREMENTE E

3. Enquanto $v[D] > P$ DECREMENTE D

4. Se $E < D$ troque $v[E]$ com $v[D]$

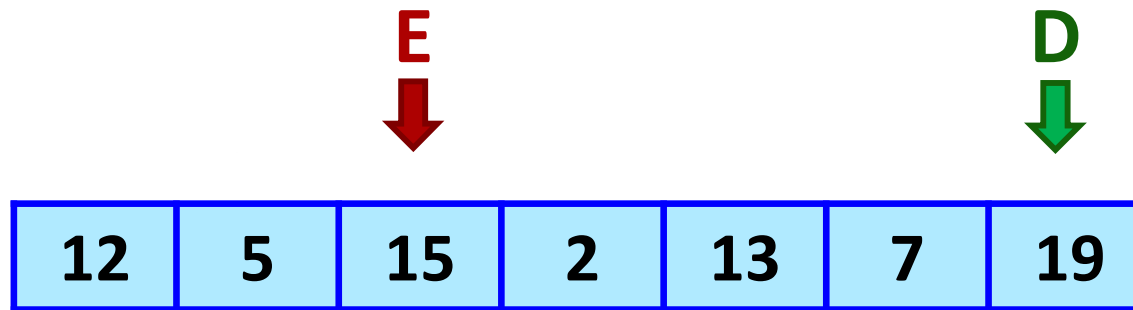
5. Repita os passos 2 – 4 até que $D \leq E$ (ou seja até se cruzarem)





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Pivô = P = 12

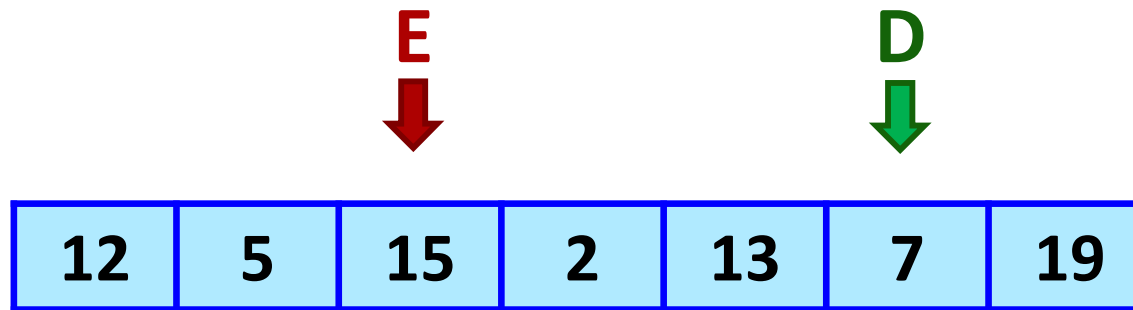
1. Escolha do pivô (**Pivô = $v[E]$**)
2. Enquanto **$v[E] \leq P$** INCREMENTE **E**
3. Enquanto **$v[D] > P$** DECREMENTE **D**
4. Se **$E < D$** troque **$v[E]$** com **$v[D]$**
5. Repita os passos 2 – 4 até que **$D \leq E$** (ou seja até se cruzarem)





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Pivô = P = 12

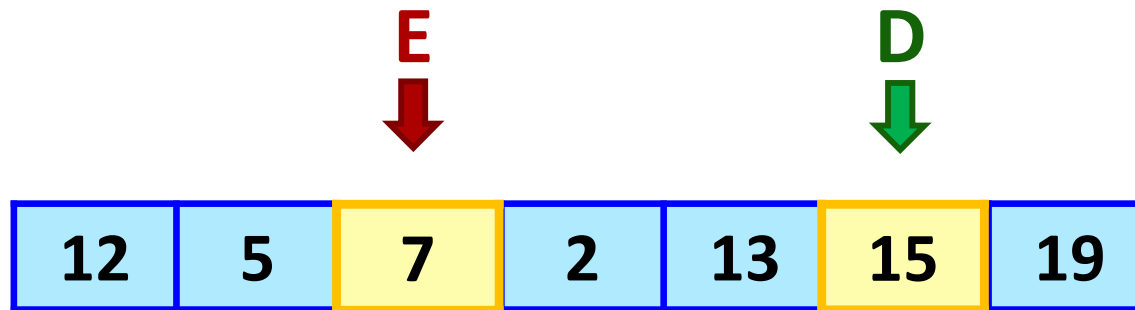
1. Escolha do pivô (**Pivô = $v[E]$**)
2. Enquanto **$v[E] \leq P$** INCREMENTE **E**
3. Enquanto **$v[D] > P$** DECREMENTE **D**
4. Se **$E < D$** troque **$v[E]$** com **$v[D]$**
5. Repita os passos 2 – 4 até que **$D \leq E$** (ou seja até se cruzarem)





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Pivô = P = 12

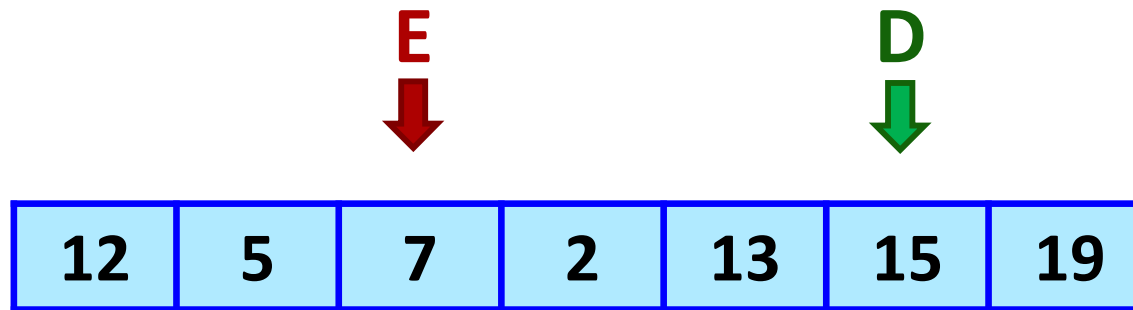
1. Escolha do pivô (**Pivô = $v[E]$**)
2. Enquanto **$v[E] \leq P$** INCREMENTE **E**
3. Enquanto **$v[D] > P$** DECREMENTE **D**
4. Se **$E < D$** troque **$v[E]$** com **$v[D]$**
5. Repita os passos 2 – 4 até que **$D \leq E$** (ou seja até se cruzarem)





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



Pivô = P = 12

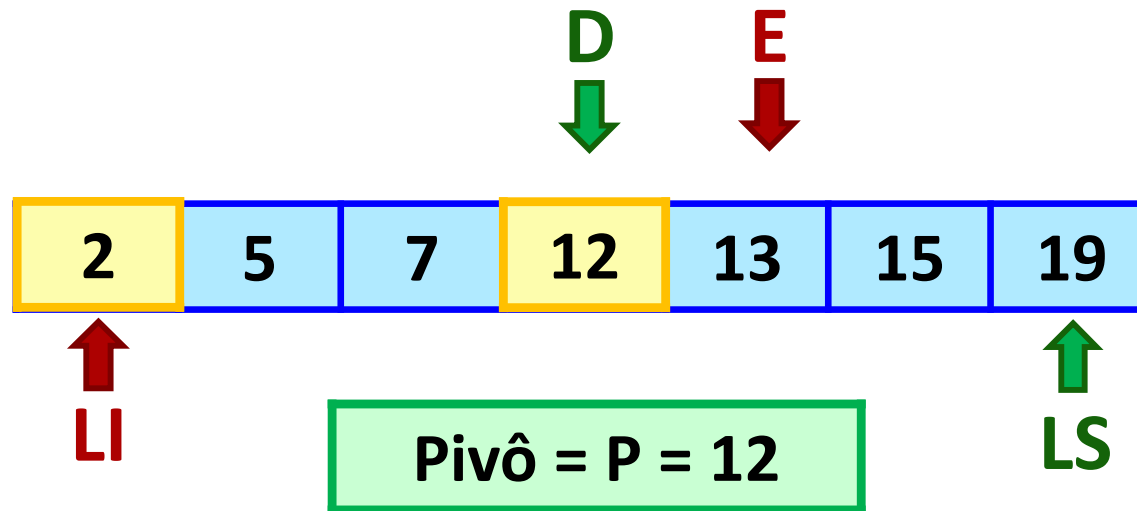
1. Escolha do pivô (**Pivô = $v[E]$**)
2. Enquanto **$v[E] \leq P$** INCREMENTE **E**
3. Enquanto **$v[D] > P$** DECREMENTE **D**
4. Se **$E < D$** troque **$v[E]$** com **$v[D]$**
5. Repita os passos 2 – 4 até que **$D \leq E$** (ou seja até se cruzarem)





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



6. Troque $X[LI]$ com $X[D]$

7. Divida o vetor e dois subvetores, a partir da Posição D e repita o processo





Métodos de Ordenação Merge Sort

- Merge, Qual a ideia?!?!?



2	5	7	12	13	15	19
---	---	---	----	----	----	----

6. Troque $X[LI]$ com $X[D]$

7. Divida o vetor e dois subvetores, a partir da Posição D e repita o processo





Métodos de Ordenação – Quick Sort

- Algoritmo com pivô sendo o elemento mais a esquerda



```
void quicksort(int *v, int LI, int LS)
{
    if(LI<LS)
    {
        int p,i;
        p = particao(v,LI,LS);
        quicksort(v,LI,p-1);
        quicksort(v,p+1,LS);
    }
}
```





Métodos de Ordenação – Quick Sort

- Algoritmo com pivô sendo o elemento mais a esquerda

```
int particao(int *v, int LI, int LS)
{
    int aux, pivô, e=LI, d=LS;
    pivô=v[e];
    while(e < d)
    {
        while((v[e]<=pivô)&& (e<LS)) { e++;}
        while((v[d]>pivô)&&(d>LI)) {d--;}
        if(e<d)
        {
            aux = v[e]; v[e]=v[d]; v[d]=aux;
        }
    }
    aux = v[LI]; v[LI]=v[d]; v[d]=aux;
    return d;
}
```



2	3	9	5	8	0	10	19			
7	1	2	3	9	5	8	0	10	19	
4	0	1	2	3	9	5	8	7	10	19
3	0	1	2	4	9	5	8	7	10	19



Métodos de Ordenação – Quick Sort

- **Vantagens**

- Melhor opção para ordenar vetores grandes
- Muito rápido por que o laço interno é simples
- Memória auxiliar para a pilha de recursão é pequena



- **Desvantagens**

- Não é estável (não se conhece uma forma eficiente para tornar o Quick Sort estável).
- Sua implementação é muito delicada e deve ser feita com cuidado.
- Um pequeno engano pode levar a efeitos inesperados para algumas entradas de dados.



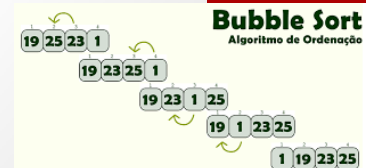


Métodos de Ordenação- Insertion Sort

- **Exercícios**



1. Dê um exemplo de um vetor com N elementos que **maximiza o número de vezes que o mínimo é atualizado no método de ordenação seleção.**
2. Mostre um exemplo de entrada que demonstra que o **método de ordenação seleção não é estável.**
3. Dado 2 sequências ordenadas de tamanho n, construa um procedimento que faça a união das mesmas. A complexidade deve ser a menor possível, mantendo a sequência resultante ordenada, e sem elementos repetidos



Bubble Sort
Algoritmo de Ordenação

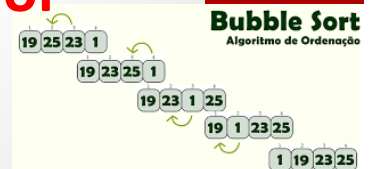


Métodos de Ordenação- Insertion Sort

• Exercícios



5. Dado como entrada, uma sequência S contendo n números reais, e um número real x . Construa um algoritmo para determinar se há dois elementos de S cuja soma é exatamente x . O algoritmo deve ser o mais eficiente possível.
6. A entrada é uma sequência de números reais, $x_1, x_2, \dots, x_n$, ordenados tal que **n é par**. Construa um algoritmo para particionar a entrada em $n/2$ pares da seguinte forma: para cada par, calculamos a soma de seus números e denotamos estas $n/2$ somas por $s_1, s_2, \dots, s_{n/2}$. O algoritmo deve encontrar o particionamento **que minimize o valor da maior soma com a melhor eficiência possível**





Métodos de Ordenação – Quick Sort



6. Modifique o Quick Sort para ordenar na forma crescente, considerando como pivô:

- O elemento mais à esquerda ($X[LI]$),
- O elemento mais à direita ($X[LS]$) e
- Elemento intermediário ($X[Meio]$)

